



DATA STRUCTURE

DS Core Group



Authors	Akhilesh Kumar Srivastava, Amit Pandey, Amrita Jyoti, Asmita Dixit, Manish Srivastava, Puneet Kumar Goyal
Authorized By	
Creation/Revision Date	1 st September 2021
Version	1.0

Chapter 7

Recursion

Table of Contents

7.1 Introduction.....	3
7.2 Functional recursion.....	4
7.3 Definition.....	5
7.4 Writing Recursive functions	5
7.4.1. Sum of N natural numbers.....	5
7.4.2 Factorial of A Given Number	6
7.4.3 Computation of Power.....	7
7.4.4 Computation of greatest common divisor (GCD).....	8
7.4.5 Finding nth Fibonacci number.....	9
7.4.6 Rope Cutting.....	10
7.5 Types of Recursion	11
7.5.1 Head and Tail Recursion.....	12
7.5.2 Tree Recursion.....	13
7.5.3 Mixed Recursion.....	14
7.5.3.1 Tower of Hanoi.....	14
7.6 Finding time and space complexity of recursive Algorithms.....	20
7.6.1 Life Cycle of Program	20
7.6.2 Process Address Space	21
7.6.2.1 Code Segment.....	21
7.6.2.2 Data Segment-	21
7.6.2.3 Stack Segment.....	21
7.6.2.4 Heap Segment.....	22
7.7 Nested Recursion	28
7.8 Indirect Recursion	29
7.9 Competitive Coding Problems	30
7.10 Exercise	32

7.1 Introduction

Let us consider the given scenario:

A child couldn't sleep, so his mother told him a story about a little frog,
Little frog couldn't sleep, so frog's mother told him a story about a little bear,
Little bear couldn't sleep, so the bear's mother told him a story about a little weasel...
who fell asleep.
... and the little bear fell asleep
... and the little frog fell asleep
... and the child fell asleep.

Let us say you have written a speech for reciting at any function. You seek the suggestions of others to make the speech perfect. You send the speech to people expert in it. If they suggest the changes, you make those changes and send the speech again to the same person for review. The process repeats until there are no changes suggested by the reviewers.

ALGORITHM Revise(Speech)

Input: A Speech text

Output: Speech text after refinement

BEGIN:

 Changes = GetFeedback(Speech);

 IF there are feedback with Changes THEN

 ApplyChanges(Speech, changes)

 Speech = Revise (Speech)

 ELSE

 RETURN Speech

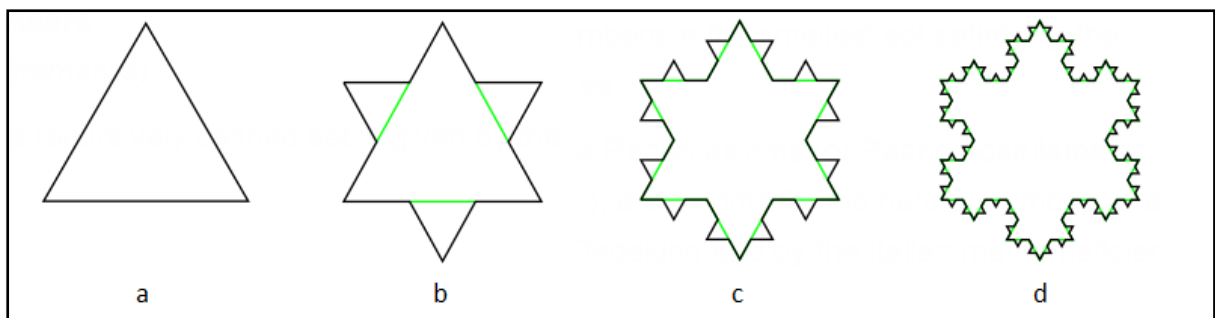
END;

Calling the same function for refinement

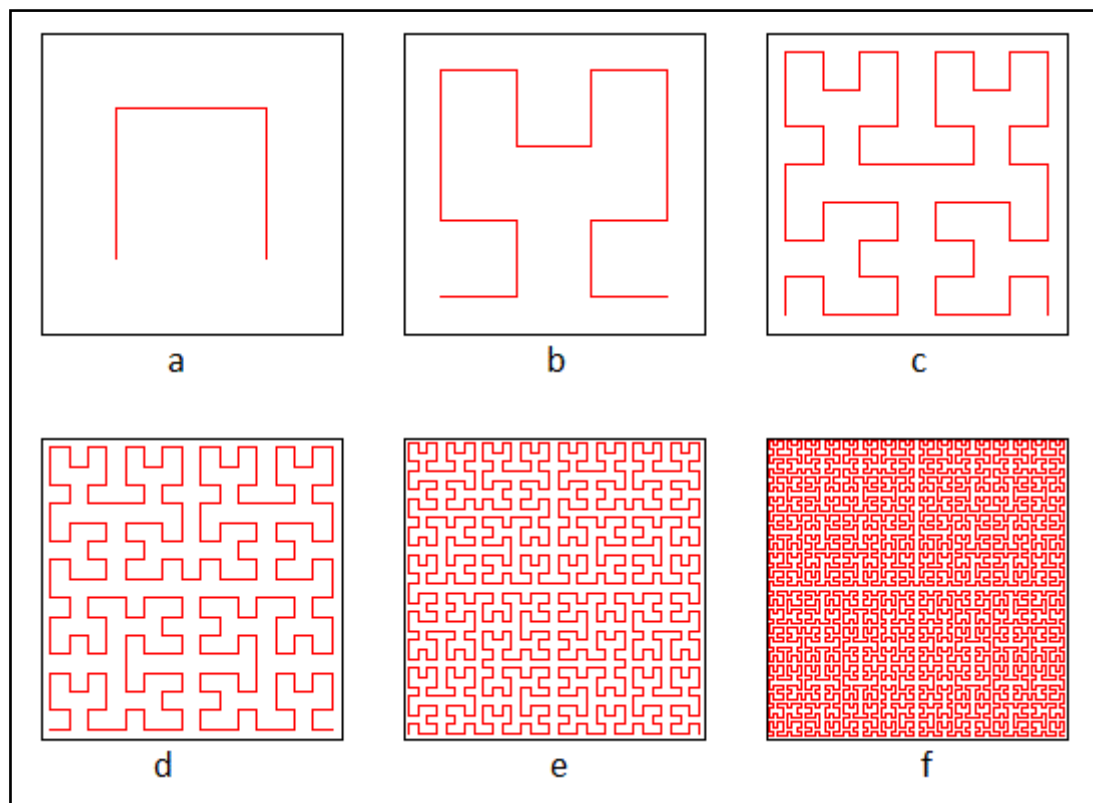
In case no refinement is there in the speech

Recursion occurs when a thing is defined in terms of itself or of its type. In the above two scenarios, we deal with a situation those are defined in their terms.

Refer to the diagrams given below showing the growth of the triangle recursively. A triangle is added on each edge of the triangle in the next iteration.



Similarly, the Hilbert space finding curve grows recursively by increasing the number of elements by 4 times in each step.



The most common application of recursion is in mathematics and computer science, where a function being defined is applied within its definition. While this defines an infinite number of instances (function values), it is often done in such a way that no infinite loop of references can occur.

In mathematics and computer science, a method exhibits recursive behavior when two properties can define it:

- A simple *base case* (or cases) — a terminating scenario that does not use recursion to produce an answer
- A *recursive step* — a set of rules that reduces all successive cases toward the base case.

7.2 Functional recursion

A function may be recursively defined in terms of itself.

- A familiar example is a Factorial number: $F(n) = n * F(n - 1)$. For such a definition to be useful, it must be reducible to non-recursively defined values (Base case): in this case $F(0) = 1$.
- Another example is the generation of power (a^n): $F(a, n) = a * F(a, n - 1)$. The function must be reducible to non-recursively defined values (Base case): in this case $F(a, 0) = 1$

- Another example is the Fibonacci number sequence: $F(n) = F(n - 1) + F(n - 2)$. For such a definition to be useful, it must be reducible to non-recursively defined values (Base case): in this case $F(0) = 0$ and $F(1) = 1$.

7.3 Definition

With respect to a programming language, recursion is defined as: “When a function calls itself (either directly or indirectly) then it is called recursive function and process is called recursion.”

Points to remember

- 1- Recursion is a problem-solving technique where the solution of larger problems is defined in smaller instances of itself.
- 2- Recursion always has a terminating condition (base condition); otherwise, it will fall in infinite loop.
- 3- Recursive function performs some part of the task and delegates the rest of it to subsequent recursive calls.

7.4 Writing Recursive functions

In this section, we will discuss the process of writing the recursive functions. However, we should first write the mathematical representation of the solution along with the base case.

7.4.1. Sum of N natural numbers

If we have to find the sum of natural numbers up to N terms, this can be written mathematically as

$$\text{Sum}(N) = \begin{cases} N + \text{Sum}(N-1) & \text{if } N > 1 \\ 1 & \text{if } N = 1 \end{cases}$$

Sum Of first N natural number = $N + \text{Sum Of first } N-1 \text{ natural number}$. When N becomes 1 Sum(1) has a fixed value 1 (Base condition). It means we are capable to define Sum in terms of Sum itself.

ALGORITHM Sum(N)

Input: Any positive number N

Output: Sum of first N natural Number

BEGIN:

```
IF N==1 THEN
    RETURN 1
```



Base case

```

        ELSE
            RETURN N+Sum(N-1) } Recursive Call
END;

```

$$\text{Sum}(N) = \begin{cases} N + \text{Sum}(N-1) & \text{if } N > 1 \\ 0 & \text{if } N = 0 \\ 1 & \text{if } N = 1 \end{cases}$$

So, in recursion, always check boundary conditions.

ALGORITHM Sum(N)

Input: Any positive number N

Output: Sum of first N natural Number

BEGIN:

```

        IF N==0 THEN
            RETURN 0 } Base case
        IF N==1 THEN
            RETURN 1 } Base case
        ELSE
            RETURN N+Sum(N-1) } Recursive Call
END;

```

7.4.2 Factorial of a given Number

Mathematically, factorial N is defined as N*Factorial of N-1.

$$\text{Factorial}(N) = \begin{cases} N * \text{Factorial}(N-1) & \text{if } N > 1 \\ 1 & \text{if } N = 0 \\ 1 & \text{if } N = 1 \end{cases}$$

Example

$$6! = 6 * 5!$$

$$5! = 5 * 4!$$

$$4! = 4 * 3!$$

$$3! = 3 * 2!$$

$$2! = 2 * 1!$$

$$1! = 1 * 0!$$

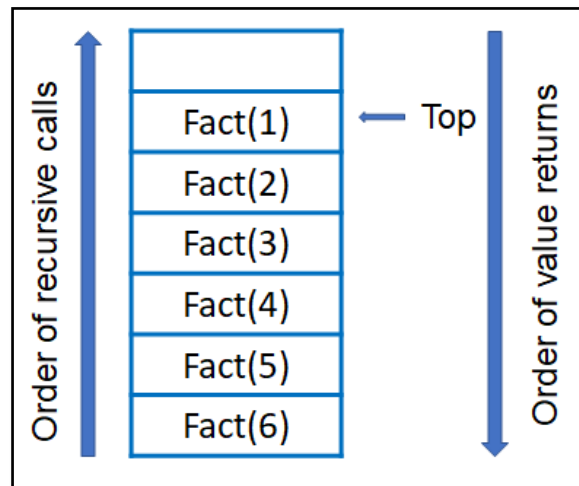
$$0! = 1 \text{ (base case)}$$

If we compute the factorial in reverse order,

$$1! = 1 * 1 = 1$$

$$2! = 2 * 1 = 2$$

$3! = 3 * 2 = 6$
 $4! = 4 * 6 = 24$
 $5! = 5 * 24 = 120$
 $6! = 6 * 120 = 720$



ALGORITHM Fact (N)

Input: Any positive number N

Output: factorial of N

BEGIN:

IF N==0 || N==1 THEN
RETURN 1

} Base case

ELSE

RETURN N* Fact (N-1)

} Recursive Call

END;

7.4.3 Computation of Power (a^N)

In the compiler, there is no arithmetic operator defined as power. Power is simulated with repeated multiplications.

$$\text{Power}(a, N) = \begin{cases} a * \text{Power}(N-1) & \text{if } N > 0 \\ 1 & \text{if } N = 1 \end{cases}$$

Example

$$3^4 = 3 * 3^3$$

$$3^3 = 3 * 3^2$$

$$3^2 = 3 * 3^1$$

$$3^1 = 3 * 3^0$$

$$3^0 = 1$$

Computing the power reverse approach

$$3^1 = 3 * 1 = 3$$

$$3^2 = 3 * 3 = 9$$

$$3^3 = 3 * 9 = 27$$

$$3^4 = 3 * 27 = 81$$

ALGORITHM Power (a, N)

Input: Base a and Power N

Output: a raised to power N

BEGIN:

IF N==0 THEN

RETURN 1

ELSE

RETURN a*Power(a, N-1)

END;

7.4.4 Computation of greatest common divisor (GCD)

Greatest common divisor of two numbers can be generated with two approaches: subtraction method and modulus method.

Subtraction method

$$\text{Gcd}(a, b) = \begin{cases} \text{Gcd}(a-b, b) & \text{if } a > b \\ \text{Gcd}(a, b-a) & \text{if } b > a \\ a & \text{if } a = b \end{cases}$$

a	b	Remarks	Action
54	16	a>b	a=a-b
38	16	a>b	a=a-b
22	16	a>b	a=a-b
6	16	b>a	b=b-a
6	10	b>a	b=b-a
6	4	a>b	a=a-b
2	4	b>a	b=b-a
2	2	a=b	Gcd is 2

ALGORITHM Gcd(a, b)

Input: Two positive numbers a and b

Output: Greatest common divisor of a and b

BEGIN:

IF a==b THEN

} Base case

```

    RETURN a
ELSE
    IF a>b THEN
        RETURN Gcd(a-b, b)
    ELSE
        RETURN Gcd(a, b-a)
END;

```

Recursive Call
Recursive Call

Modulus method

$Gcd(a, b) = \begin{cases} Gcd(a\%b, b) & \text{if } a>b \\ Gcd(a, b\%a) & \text{if } b>a \\ a & \text{if } b=0 \\ b & \text{if } a=0 \end{cases}$

a	b	Remarks	Action
54	16	a>b	a=a%b
6	16	b>a	b=b%a
6	4	a>b	a=a%b
2	4	b>a	b=b%a
2	0	b=0	Gcd is 2

ALGORITHM Gcd(a, b)

Input: Two positive numbers a and b

Output: Greatest common divisor of a and b

BEGIN:

```

    IF a==0 THEN
        RETURN b
    ELSE
        IF b==0 THEN
            RETURN a
        ELSE
            IF a>b THEN
                RETURN Gcd(a%b, b)
            ELSE
                RETURN Gcd(a, b%a)
        END;
    END;

```

Base case
Base case
Recursive Call
Recursive Call

7.4.5 Finding nth Fibonacci number

Fibonacci series is given as: 0 1 1 2 3 5 8 13 21 Here sum of previous two terms results in the next term. Mathematically, finding nth term can be represented recursively as:

$Fib(n) = \begin{cases} Fib(n-1) + Fib(n-2) & \text{if } n>2 \\ 0 & \text{if } n=1 \end{cases}$

1

if $n=2$

Let us represent Fibonacci series diagrammatically

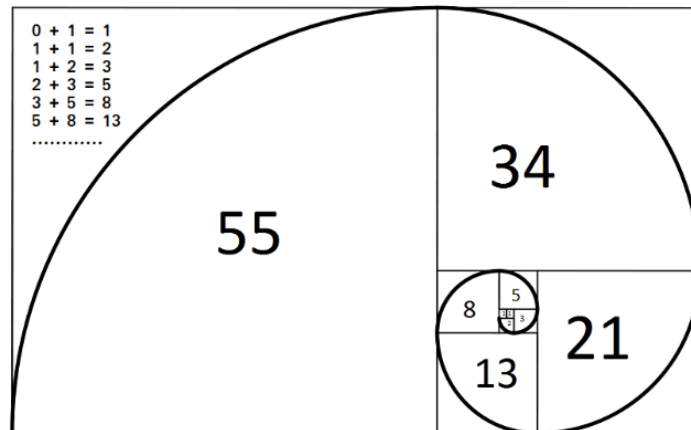


Figure: Generation of Fibonacci Series

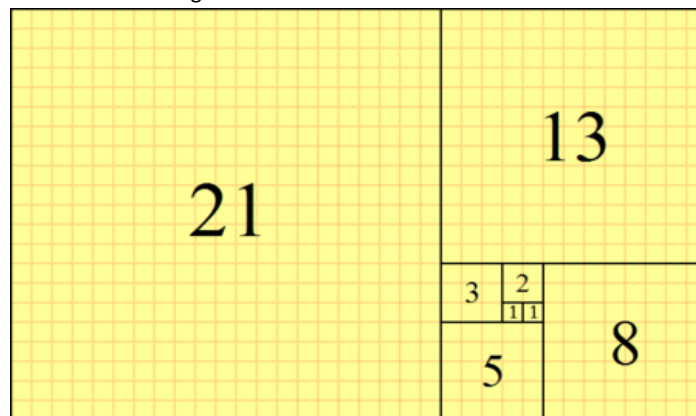


Figure: Creation of Square blocks of size equal to Fibonacci term

ALGORITHM Fib(N)

Input: Any positive number N

Output: N^{th} Fibonacci term

BEGIN:

IF $N==1$ THEN

RETURN 0

IF $n==2$ THEN

RETURN 1

ELSE

RETURN $\text{Fib}(N-1) + \text{Fib}(N-2)$

END;

7.4.6 Rope Cutting

Consider a rope of a certain size N is given. Ropes can be cut in sizes a, b and c. We need to find the maximum number of pieces that can be obtained with sizes a, b and c.

Example 1:

$N=5$, $a=2$, $b=1$, $c=5$

Maximum pieces are 5 of size 1 each

Example 2:

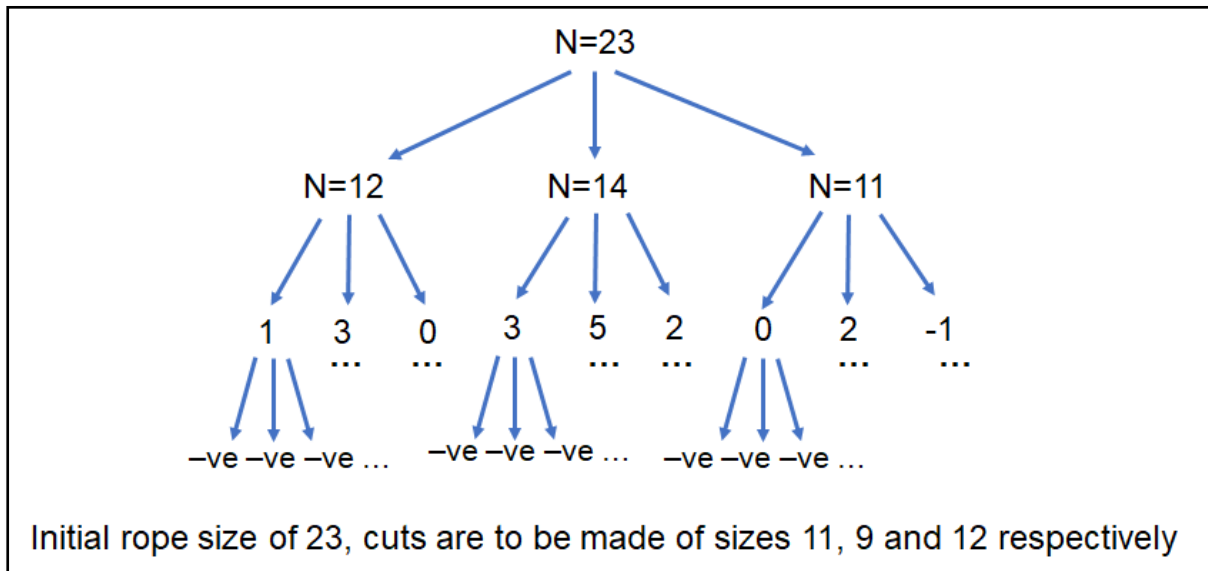
$N=23$, $a=12$, $b=14$, $c=11$

Maximum pieces are 2 of size 11 and 12

Example 3:

$N=23$, $a=12$, $b=14$, $c=16$

No combination of these sizes, so the rope cutting is not possible.



ALGORITHM RopeCut(N , a , b , c)

BEGIN:

IF $N==0$ THEN
 RETURN 0

} Base case

IF $N<0$ THEN
 RETURN -1

} Base case

ELSE

$r = \text{Max}(\text{RopeCut}(N-a, a, b, c), \text{RopeCut}(N-b, a, b, c), \text{RopeCut}(N-c, a, b, c))$

} Recursive Call

IF $r==1$ THEN
 RETURN -1

ELSE
 RETURN $r+1$

END;

7.5 Types of Recursion

1. Head Recursion
2. Tail Recursion
3. Tree Recursion
4. Mixed Recursion

7.5.1 Head and Tail Recursion

In this type of recursion, the Recursive function performs some task and calls itself. If the call is made before function performs its task, then it is called head recursion. On the other hand, if the function performs its task first and is followed by a recursive call, then it is tail recursion.

Example of Head recursion

ALGORITHM Fun(N)

Input: Any positive number N

Output: print number N to 1

BEGIN:

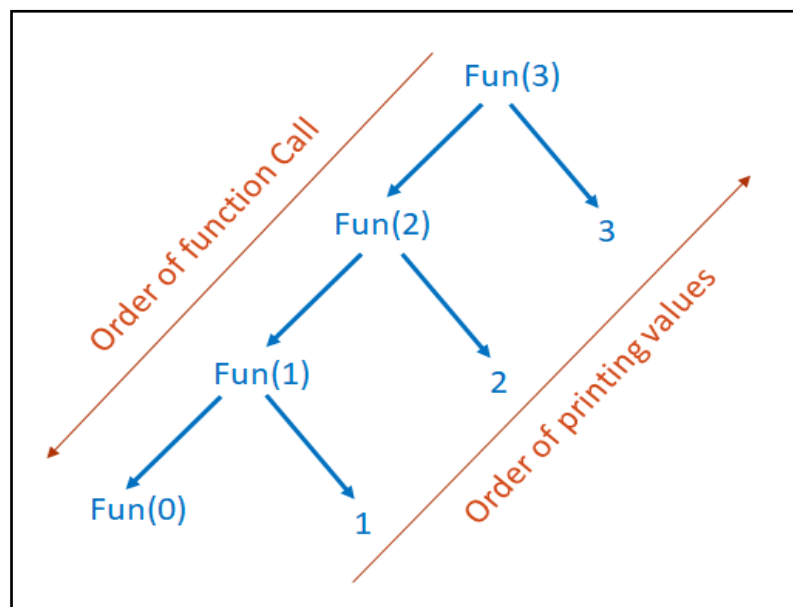
IF N>0 THEN

Fun(N-1)

WRITE("N")

} Recursive Call

END;



Example Tail recursion

ALGORITHM Fun(N)

Input: Any positive number N

Output: print number N to 1

BEGIN:

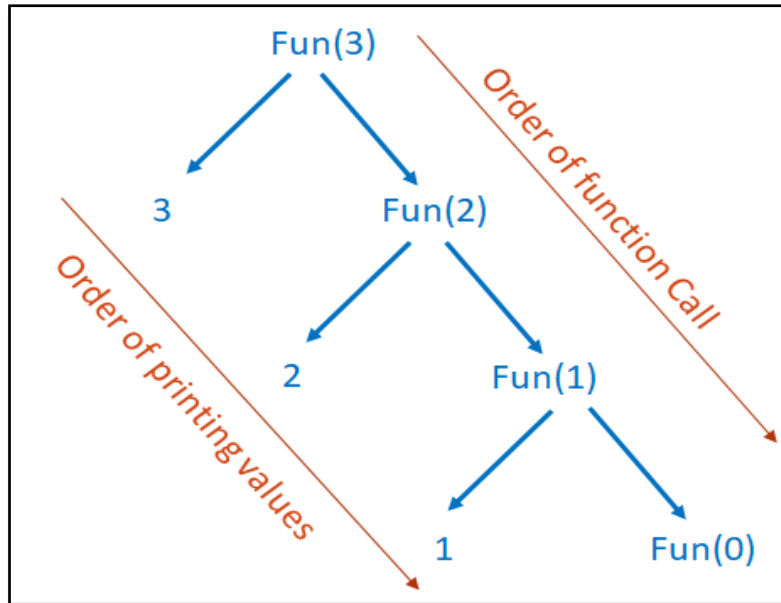
IF N>0 THEN

WRITE("N")

Fun(N-1)

} Recursive Call

END;



7.5.2 Tree Recursion

The tree recursion is a type of recursion in which recursive calls grow in the form of a tree. Let us recap the generation of nth Fibonacci term.

ALGORITHM Fib(N)

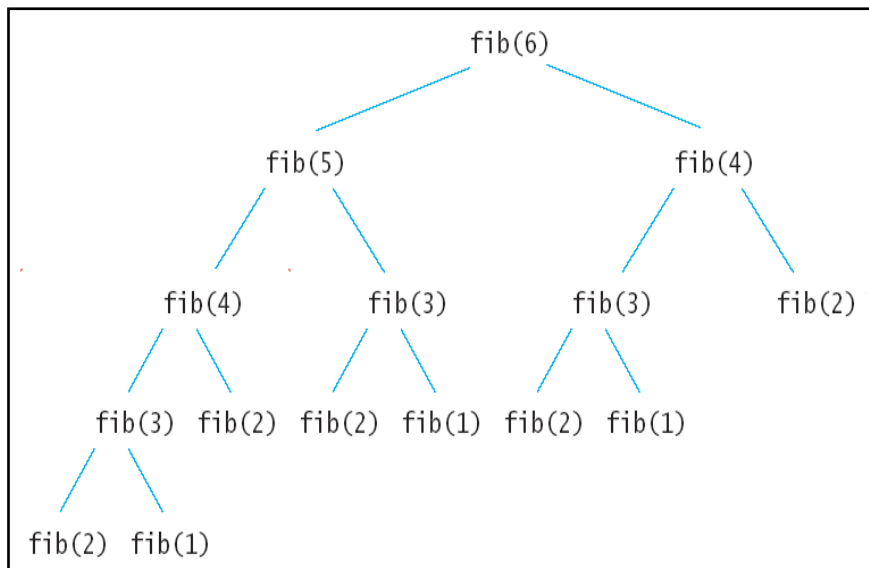
Input: Any positive number N

Output: Nth Fibonacci term

BEGIN:

IF N == 1 THEN	}	Base case
RETURN 0		
IF N == 2 THEN	}	Base case
RETURN 1		
ELSE	}	Recursive Call
RETURN Fib(N-1) + Fib(N-2)		

END;



7.5.3 Mixed Recursion

Mixed recursion is the recursion that utilizes the concept of Head, Tail and Tree recursion. To understand the concept of Mixed Recursion, let us take an interesting concept of Towers of Hanoi.

7.5.3.1 Tower of Hanoi

According to Hindu Mythology, A Rig-Veda a story says: Devtas (देवता) and Danavs (दानव) used to reside side by side wherein the Danavs used to create problems for Devtas. Devtas requested Lord Shiva to create the universe. Lord Shiva requested Lord Brahma the same. Lord Brahma created the universe by establishing three corners. Aakash (आकाश), Paatal(पाताल), and Dharti(धरती) Lok. Akash Lok was given to Devtas, Paatal to Danavs and Dharti to newly created creatures the human beings. The first City created on the earth was Kashi. Lord Brahma established three towers in Aakash, Paatal, and Dharti Lok and 64 disks of reducing the size (from bottom to top) on stack-like arrangements at Tower in Aakash Lok. The Devta was given the task of shifting disks from Aakash Lok tower to Paatal Lok, taking Dharti Lok tower as a mediator.

Story about an Indian temple Kashi Vishwanath in Varanasi, is also of fame. This says the temple contains a big chamber with three time-worn posts in it, surrounded by 64 golden disks. Brahmin priests, acting out the command of an ancient insight, have been moving these disks in accordance with the immutable rules of Brahma since that time. The puzzle is therefore also known as the Tower of Brahma puzzle.

According to the legend, when the last move of the puzzle is completed, the world will end. If the legend were true, and if the priests were able to move disks at a rate of one per day, using the smallest number of moves, it would take them $2^{64} - 1$ days (Trillian years).

A similar arrangement is made in the Buddhist temple in Hanoi (Vietnam). The Monk makes one move per day according to the rule that a bigger disk cannot be kept above the smaller one. The puzzle is named under the Hanoi and hence known as towers of Hanoi.

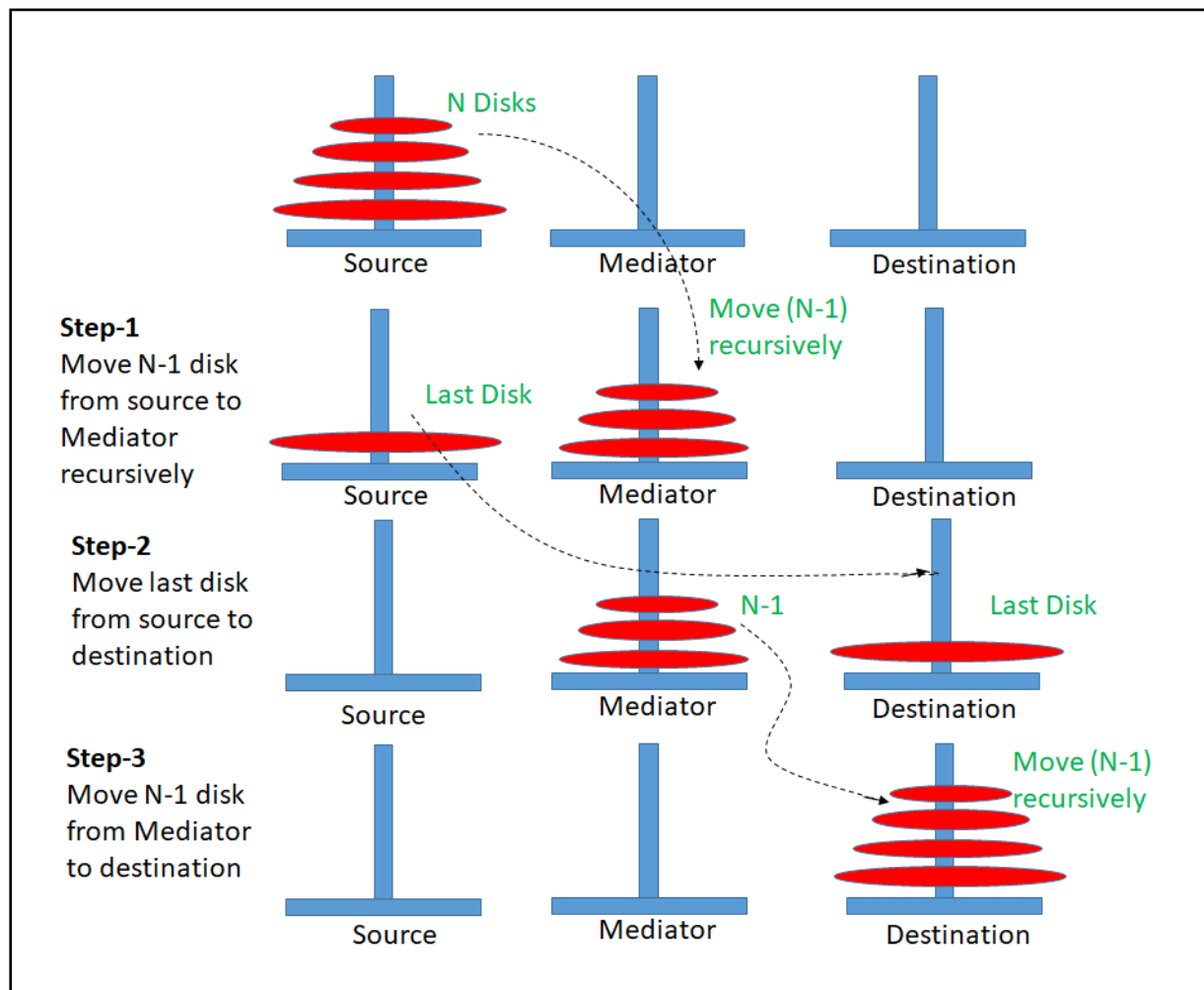
- There are three Pegs: Source, Destination and Mediator and there are n disks (of different sizes). All disks are initially at source (in the order of bigger to smaller).
- Our aim is to move all disks from Source to Destination using Mediator but never place a larger disk on the top of a smaller one.
- At a time, only one move is allowed.

Logic-

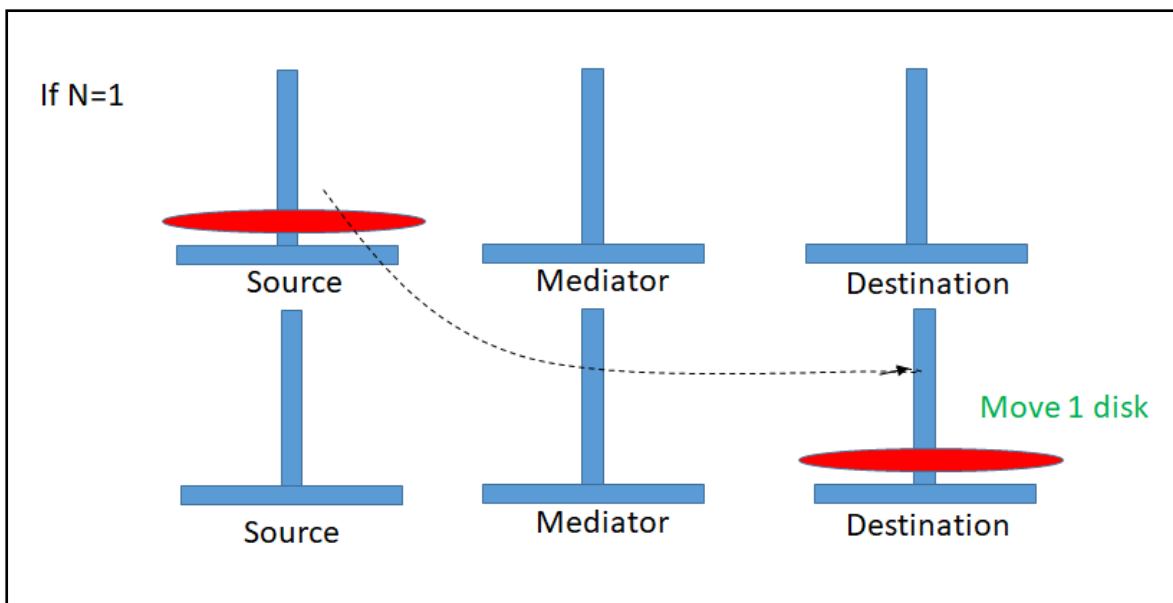
1-Move $n-1$ disk source to Mediator using Destination.

2-Move n^{th} disk from Source to Destination.

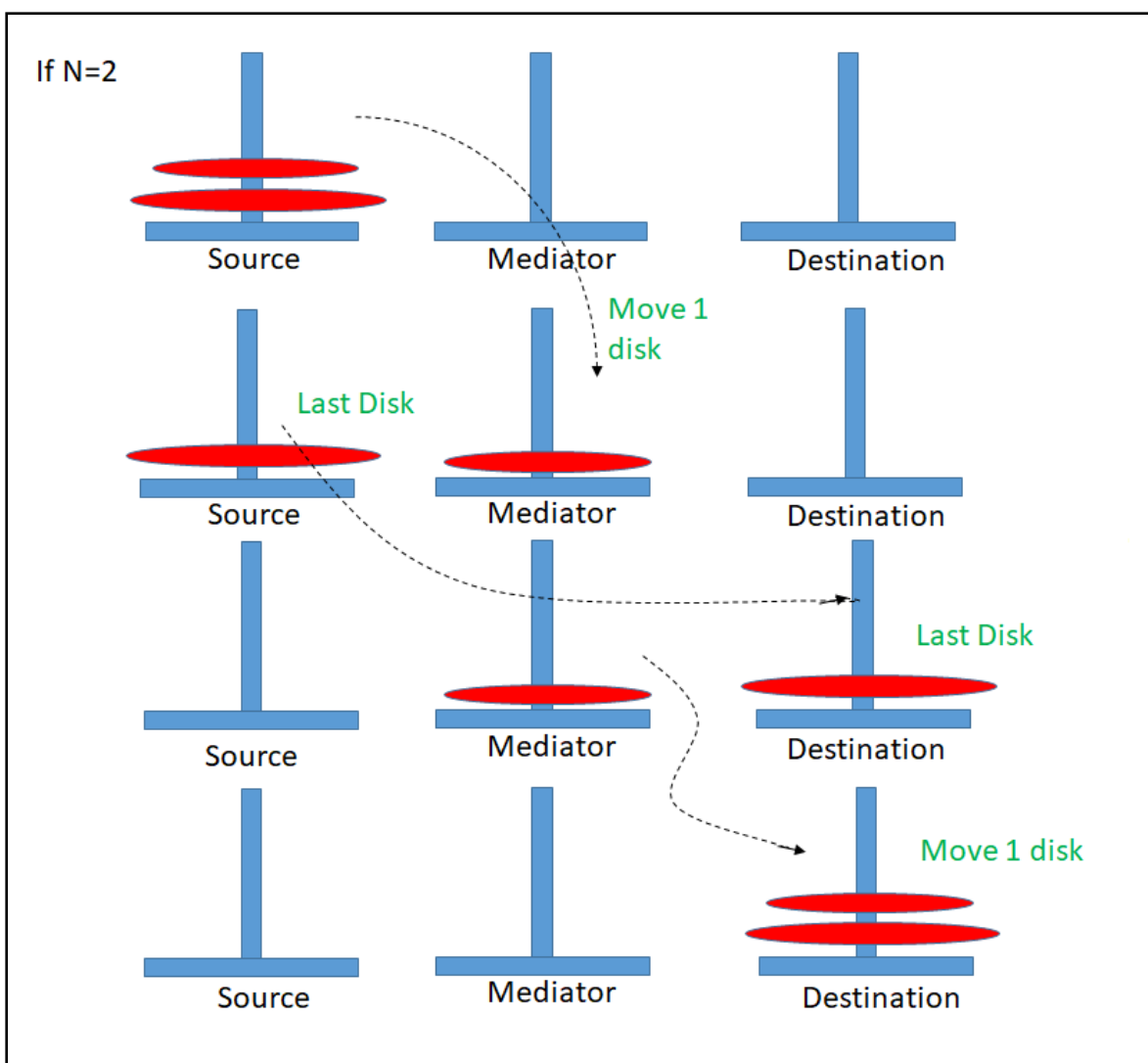
3-Move $n-1$ disk from Mediator to Destination using source



If there is only 1 disk at source Peg



If there are 2 disks at source Peg

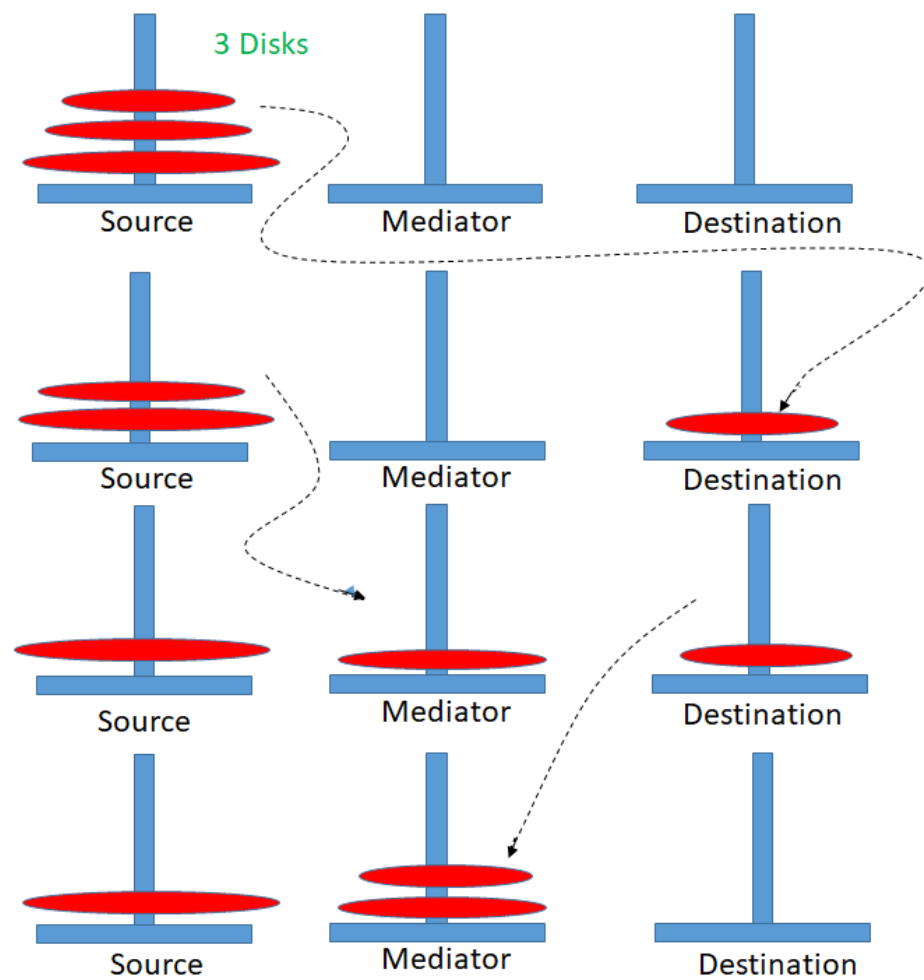


If there are 3 disks at source Peg

If $N=3$

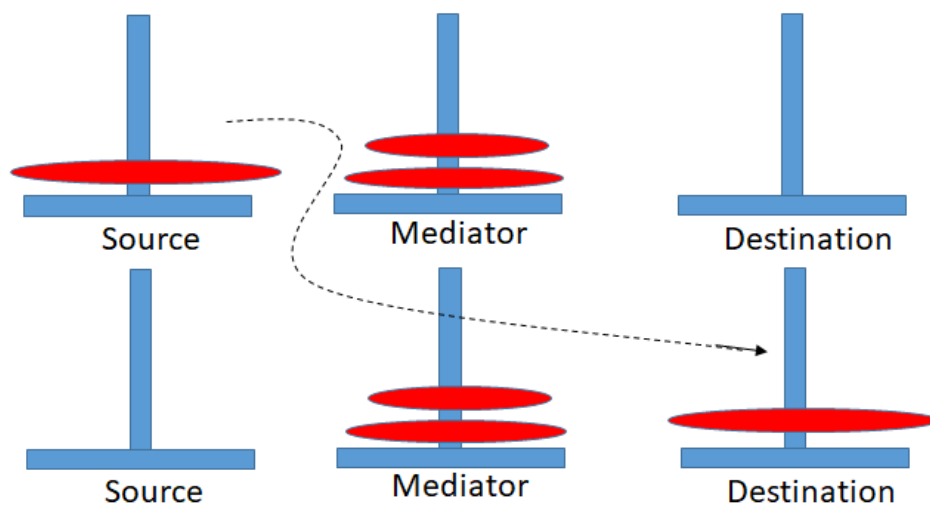
Step-1

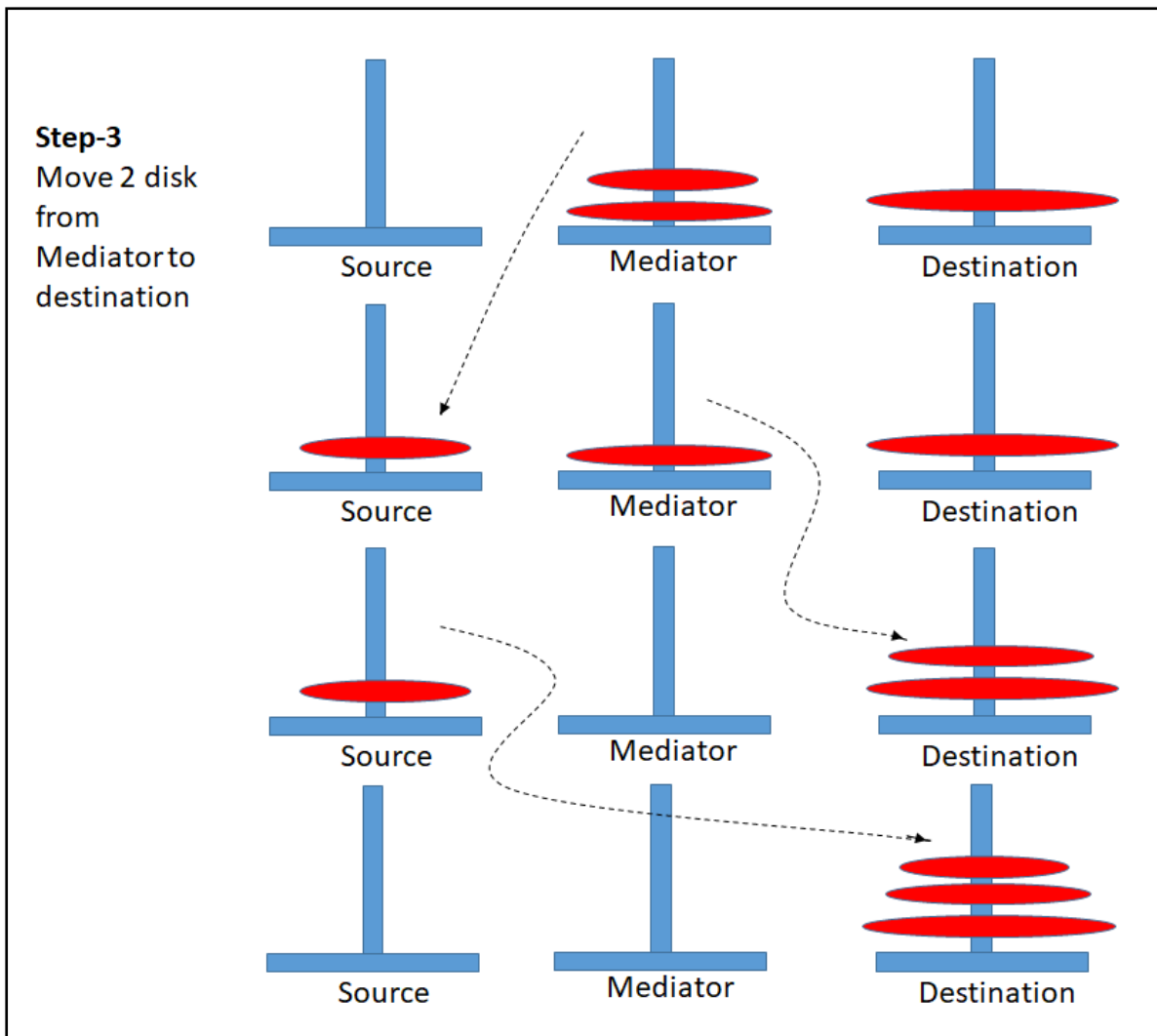
Move 2 disk
from source
to Mediator
recursively



Step-2

Move last
disk from
source to
destination





ALGORITHM ToH (S, M, D, n)

Input: A number n and three characters representing Source, Mediator, Destination Peg

Output: Disk movement from peg to peg

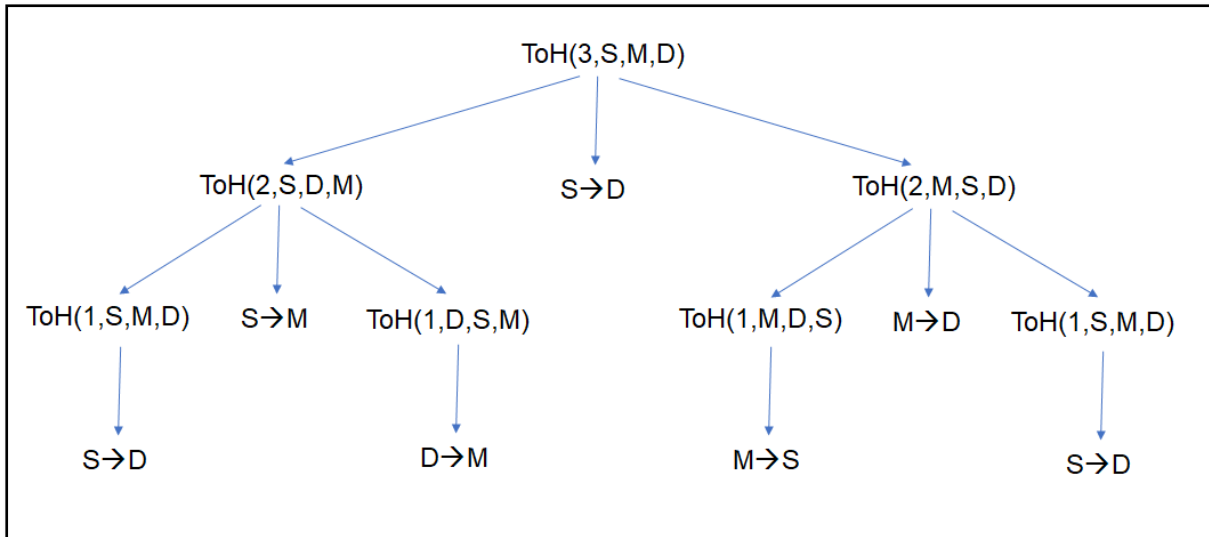
BEGIN:

IF n == 1 THEN	}	Base case
RETURN		
ToH(S, D, M, n-1)	}	Recursive Call

WRITE("Move disk from Source or Destination")

ToH(M, S, D, n-1)	}	Recursive Call

END;



Sequence of steps performed:

1. $S \rightarrow D$ 2. $S \rightarrow M$ 3. $D \rightarrow M$ 4. $S \rightarrow D$ 5. $M \rightarrow S$ 6. $M \rightarrow D$ 7. $S \rightarrow D$

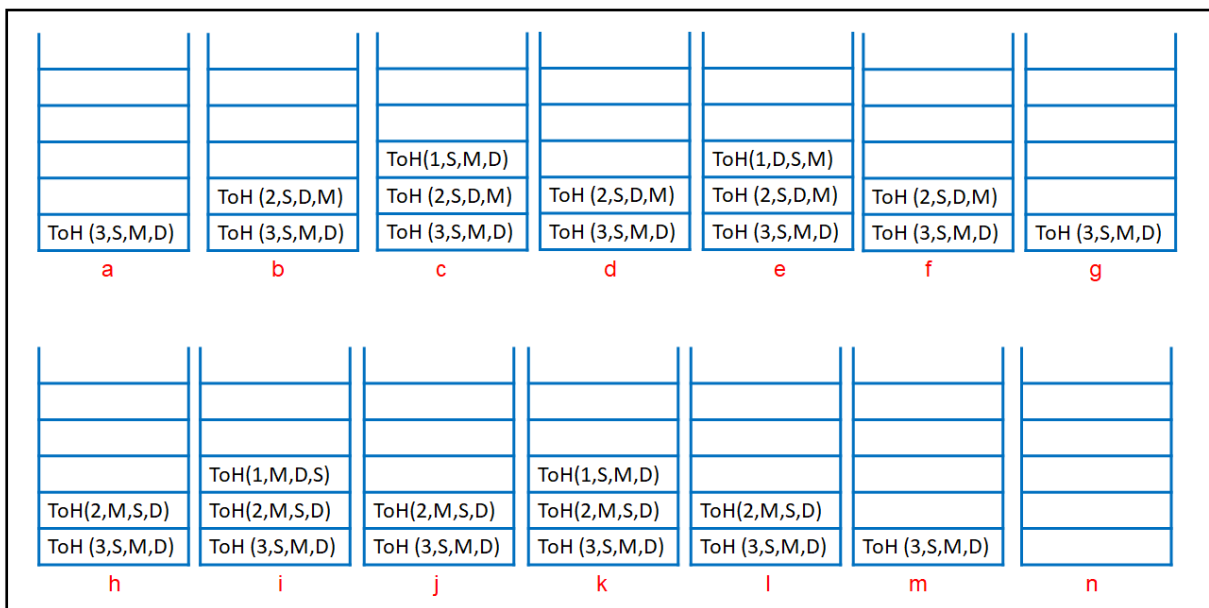
Total Function Calls : 7

Total Push in Call Stack : 7

Total Pop in Call Stack : 7

For N disks on source peg, there will be $2^N - 1$ function calls. Therefore, total Push and Pop operations will be $2 * (2^N - 1)$ i.e., $2^{N+1} - 2$. Since Push and pop operations take constant time, the Time complexity will be $\Theta(2^N)$.

To understand the space complexity, let us see the maximum number of pending activation records at any moment.



It can be seen from the diagram above that the maximum pending activation records at any moment are 3 (if the number of disks initially on the source peg is 3). If the disks are N , the figure would be N . Considering constant space for each activation record and the space complexity would be $\Theta(N)$.

7.6 Finding time and space complexity of recursive Algorithms

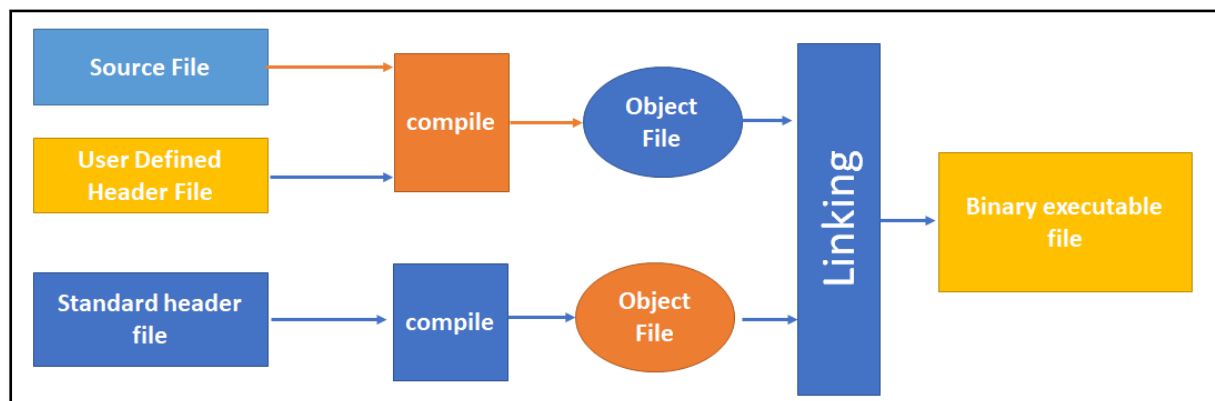
Before understanding the method to find the time and space complexity of recursive algorithms, let us first understand the life cycle of the program and process address space.

7.6.1 Life Cycle of Program

For the execution of a program, several steps are involved.

1. Source file (program) and the user-defined header files are compiled to generate the object file.
2. Standard header files are also compiled to generate the object file
3. Object files in steps 1 and 2 are linked together to generate the binary executable file.
4. Executable file is loaded in the main memory.
5. During the program's execution, dynamically linked library files are linked with the executing process as per the requirement.

Area of memory where the process is loaded is called process address space.



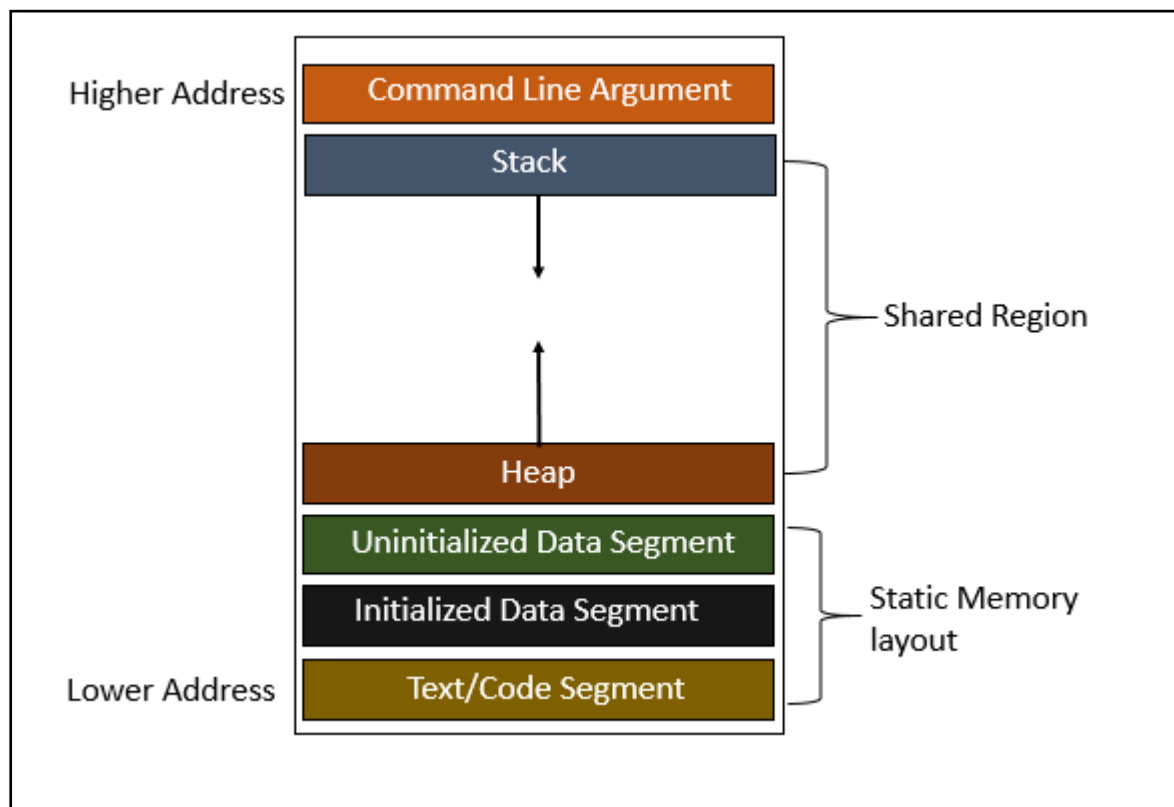
With respect to a C program, following are the extensions for various kinds of the files.

Program.c → Source File

Program.o → Object File

Program.exe → Binary Executable File

7.6.2 Process Address Space



Process address space has the following segments

- Code Segment
- Data Segment
- Stack
- Heap

7.6.2.1 Code Segment

- This segment contains the machine code of compiled program.
- It is read-only and can't be changed when the program is executing.
- Size of code segment is fixed during load time.

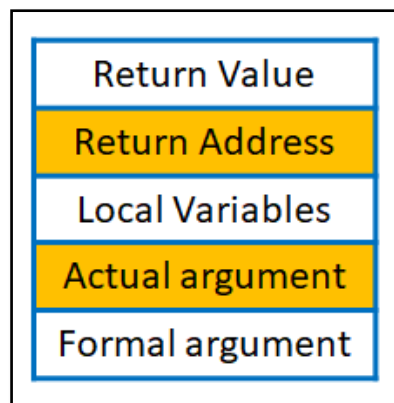
7.6.2.2 Data Segment-

- All global and static variables are allocated in this segment during load time.
- All load time variables are initialized at the load time.
- The size of data segment is fixed at load time and does not change when the program is executing.

7.6.2.3 Stack Segment

The stack segment contains activation records called stack frames of all active functions. An active function is a function that is currently under the call.

An activation record contains the following elements.

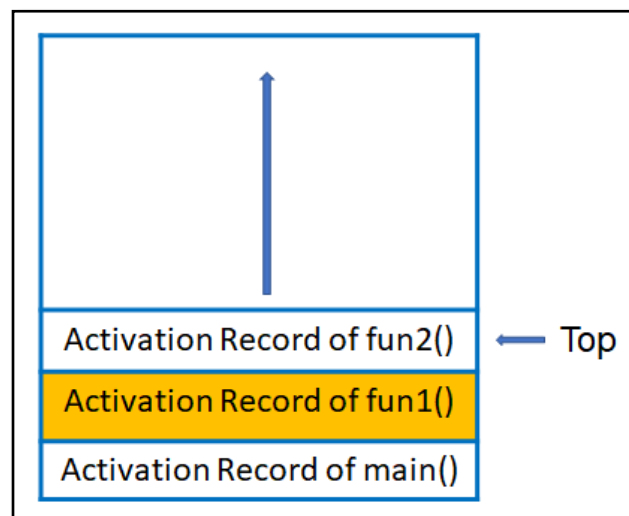


7.6.2.4 Heap Segment

For all the dynamically allocated memory, the space reserved is the Heap Segment.

Example of stack segment (C Language Program)

```
void main()
{
    fun1();
}
/*****/
void fun1(void)
{
    fun2()
}
/*****/
void fun2(void)
{
    printf("hello");
}
/*****/
void fun3(void)
{
}
/*****/
```



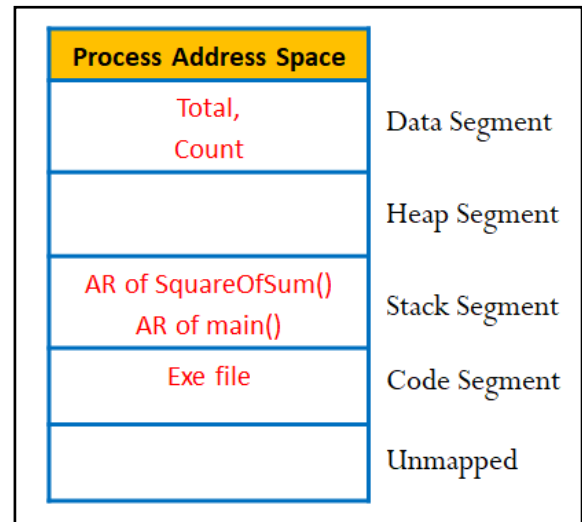
The activation record of main is created first in the stack segment. Main calls fun1; hence its activation record is formed next. fun1 calls fun2; hence the next activation record is formed for fun2, which is the top of the stack. Since fun3 is never called, so its activation record is not formed.

Note: An activation record is formed only if the function is called.

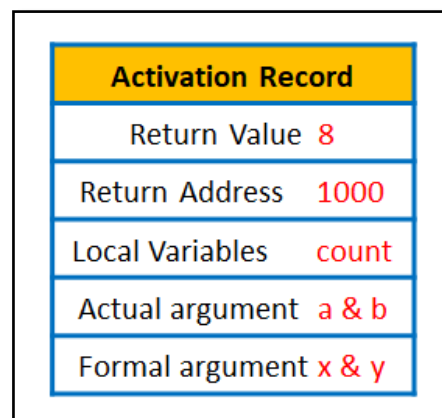
Memory allocation for different segments through example (C Language)

```
int SquareOfSum(int x,int y);
int total;

int main()
{
    int a=4, b=2;
    total=squareofsum(a,b);
    printf("square of sum=%d\n", total);
    return 0;
}
```



```
int squareofsum(int x,int y)
{
    int count=0;
    printf("fun called %d times",++count);
    return (x*y);
}
```



Complexity of Recursive Functions

Upon each recursive call, the push of activation record takes place in the stack segment. Every return will cause the Pop of activation records. As push takes constant time and pop, total time complexity will be equal to the number of push and pop operations taking place.

Because of recursive calls, activation records are pushed into a stack segment that occupies some space in the memory. Let us consider that the space taken by an activation record is constant. Space complexity can be found by finding the maximum height of the stack or maximum pending activation records at any moment.

Let us revisit the recursive functions defined till now to compute their time and space complexity.

1. Factorial

ALGORITHM Factorial(N)

Input: Any positive number N

Output: factorial of N

BEGIN:

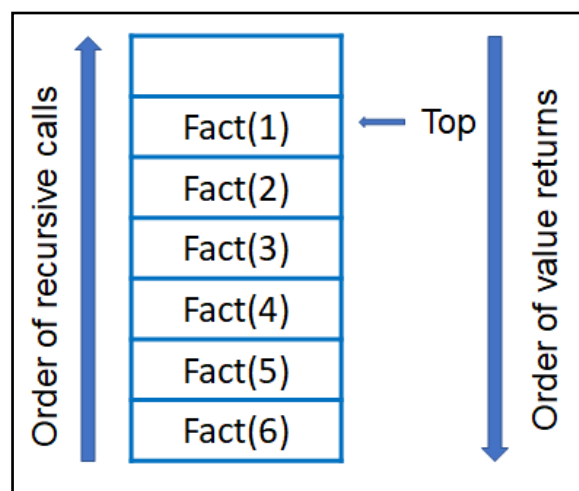
IF $N==0$ || $N==1$ THEN

RETURN 1

ELSE

RETURN $N*\text{fact}(N-1)$

END;



Time Complexity: Total Number of function calls is n ; hence total push and pop operations would be $2*n$. Time complexity of this function would be $\Theta(n)$.

Space Complexity: Space complexity will be equal to number of maximum pending activation records, n for this function i.e., $\Theta(n)$.

2. Computation of Power

ALGORITHM Power (a, N)

Input: Base a and Power N

Output: a raised to power N

BEGIN:

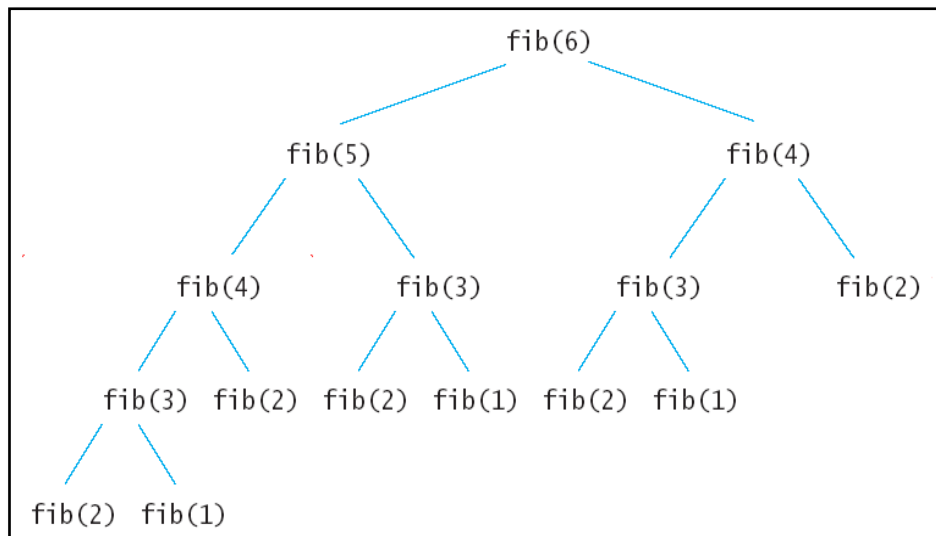
IF $N==0$ THEN

RETURN 1

ELSE

RETURN $a*\text{Power}(a, N-1)$

END;



For n=1 Total calls = 1

For n=2 Total calls= 1

For n=3 Total calls = 3

For n=4 Total function calls =5

For n=5 Total function call = 9

For n=6 Total function call = 15

For n=7 Total function call = 25

For n=8 Total function call = 41

Number of function calls can be found with a direct formula **$2 * Term_{N+1} - 1$**

- For example, N= 5,
The number of function call is $2 * Term_{5+1} - 1$
 $= 2 * Term_6 - 1$
 $= 2 * 5 - 1$
 $= 9$
- For example, N= 8,
The number of function call is $2 * Term_{8+1} - 1$
 $= 2 * Term_9 - 1$
 $= 2 * 21 - 1$
 $= 41$

If we write the recurrence for the recursive calls here,

$$T(N) = T(N-1) + T(N-2) + 1$$

1 is representing the effort for condition check for base condition

After solving this, we get $\Theta(2^N)$

Time Complexity = $\Theta(2^N)$

Taking the example of 6th Fibonacci term, maximum pending activation records is Equal to 5.
For N, it will be N-1.

Space Complexity = $\Theta(N)$

Time complexity of $\Theta(2^N)$ is not acceptable as this is exponential. As the term increases, the effort increases exponentially. This is happening because of the redundant calls to functions. E.g. in the above diagram, number of calls for different values of n is given as:

Fib(5) = 1

Fib (4) = 2

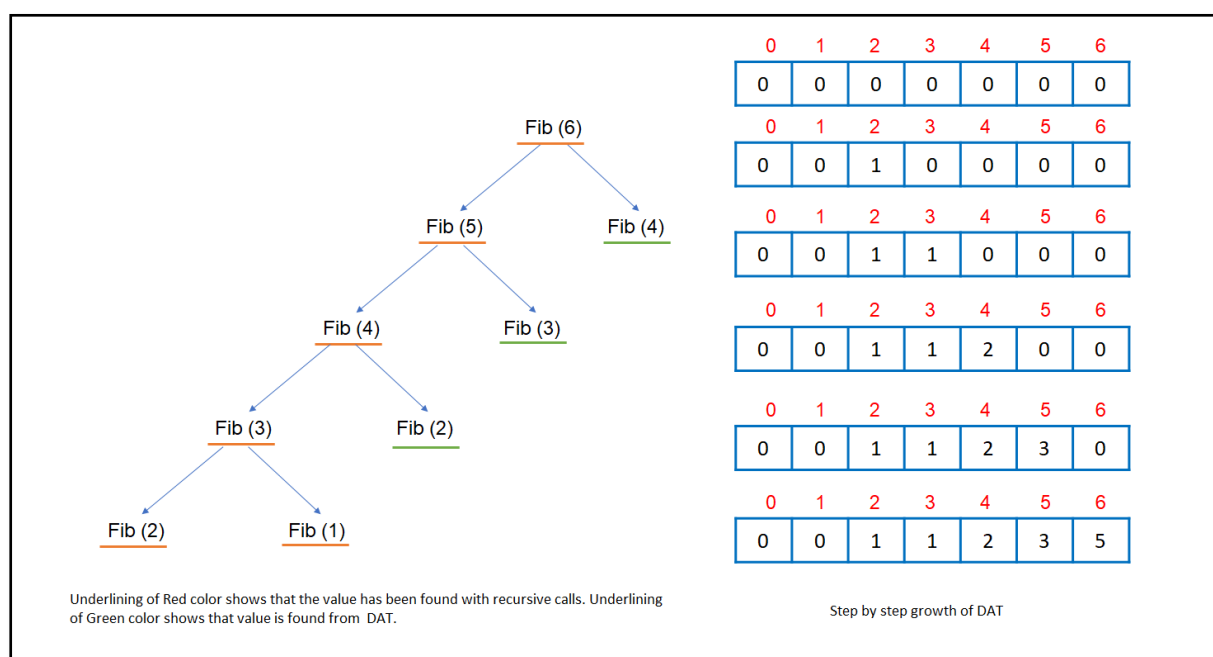
Fib (3) = 3

Fib (2) = 5

Fib(1) = 3

The redundant calls can be reduced by saving the term in some storage and referring to that term from the storage itself whenever there is a call for that term. The direct address table discussed in chapter 4 will help us in this. If we have to find nth Fibonacci term, the size of direct address table hence taken will be n+1. All values of this table should be initialized to 0. Value 0 in various cells of DAT represents the term that has not been found. The non-zero value represents that term has been found.

Example: Let us say we have to find the 6th Fibonacci term. A DAT (Direct Address Table) of size 7 will take all values initialized as zero. This DAT can be referred by recursive function in case the term has not been discovered yet.



ALGORITHM Fib(N, DAT[])

Input: Any positive number N and a DAT

Output: Nth Fibonacci term

BEGIN:

```
    IF N==1 THEN
        DAT[1] = 0
    IF N==2 THEN
        DAT[2] =1
    ELSE
        IF(DAT[N] != 0 THEN
            RETURN Fib(N-1) + Fib(N-2)
        RETURN DAT[N]
```

END;

In the above, the function calls are less than the previous version of the function. Total of 9 calls is there for 6th Fibonacci term. 11 calls for 7th term, 13 calls for 8th term and so on so forth. It can be concluded that the complexity is $\Theta(N)$. Space Complexity remains $\Theta(N)$ as for the previous version.

7.7 Nested Recursion

When a function calls itself with call of itself as a parameter, this is called nested recursion. See the function given below:

ALGORITHM A()

BEGIN:

```
    A(A())
```

END;

Function A() calls itself with the parameter in the function as A() itself.

Ackermann function

Ackermann function, named after Wilhelm Ackermann, is one of the simplest and earliest-discovered examples of a not primitive recursive.

$$A(m, n) = \begin{cases} n+1 & \text{if } m=0 \\ A(m-1, 1) & \text{if } n=0 \\ A(m-1, A(m, n-1)) & m>0, n>0 \end{cases}$$

Example: Calculate A(1, 2)

$A(1,2) = A(0, A(1, 1))$

$$\begin{aligned}
&= A(0, A(0, A(1, 0))) \\
&= A(0, A(0, A(0, 1))) \\
&= A(0, A(0, 2)) \\
&= A(0, 3) \\
&= 4
\end{aligned}$$

Example: Calculate A(1,5)

Answer=7

$$\begin{aligned}
A(1,5) &= A(0, A(1,4)) \\
&= A(0, A(0, A(1,3))) \\
&= A(0, A(0, A(0, A(1,2)))) \\
&= A(0, A(0, A(0, A(0, A(1,1))))) \\
&= A(0, A(0, A(0, A(0, A(0, A(1,0)))))) \\
&= A(0, A(0, A(0, A(0, A(0, A(0, 1)))))) \\
&= A(0, A(0, A(0, A(0, A(0, 2)))))) \\
&= A(0, A(0, A(0, A(0, 3)))) \\
&= A(0, A(0, A(0, 4))) \\
&= A(0, A(0, 5)) \\
&= A(0, 6) \\
&= 7
\end{aligned}$$

7.8 Indirect Recursion

When any function calls itself indirectly through another function, it is termed as indirect recursion. In the example given below, function A() indirectly is called from function B() and function B() is indirectly called from function A().

ALGORITHM A(n)

Input: Any positive number n

Output: input integer

BEGIN:

```

IF n<=1 THEN
    RETURN
ELSE
    B(n-2)
    WRITE("n")
    B(n-1)

```

END;

ALGORITHM B(n)

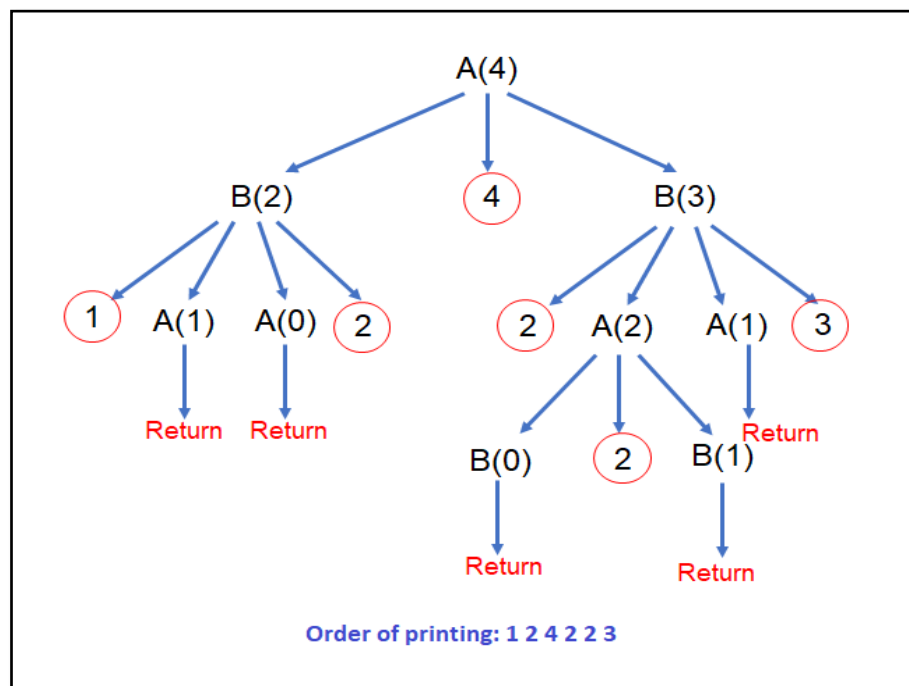
Input: Any positive number n

Output: input integer

BEGIN:

```
IF n<=1 THEN
    RETURN
ELSE
    WRITE(n-1)
    A(n-1)
    A(n-2)
    WRITE(n)
```

END;



7.9 Competitive Coding Problems

Permutation Problem (Recursion within loop)

A permutation also called an "arrangement number" or "order," is a rearrangement of the elements of an ordered list S into a one-to-one correspondence with S itself. A string of length n has $n!$ permutation.

Below are the permutations of string ABC.

ABC ACB BAC BCA CBA CAB

ALGORITHM Permutation(A[], n, l)

BEGIN:

```
IF l == n-1 THEN
    display(A, n)
```

```

        RETURN
    FOR i=l TO n DO
        Swap(A[i], A[l])
        Permutation(A,n,l+1)
        Swap(A[i], A[l])
END;
```

Problem 2:

Given an array of integers, write a recursive code that add sum of all the previous numbers to each index of array.

Input: 1,2,3,4,5,6,7

Output: 1,3,6,10,15,21,28

ALGORITHM CumulativeSum(A[], N)

BEGIN:

```

    If N==1 THEN
        RETURN A[0]
    ELSE
        A[N-1] = CumulativeSum(A, N-1) + A[N-1]
        RETURN A[N-1]
```

END;

$A[6] = CS(A[], 5) + A[5]$

$A[5] = CS(A[], 4) + A[4]$

$A[4] = CS(A[], 3) + A[3]$

$A[3] = CS(A[], 2) + A[2]$

$A[2] = CS(A[], 1) + A[1]$

$A[1] = CS(A[], 0) + A[0]$

A[0] Direct return because of base condition

Computation in reverse order

$A[1] = 2 + 1 = 3$

$A[2] = 3 + 3 = 6$

$A[3] = 6 + 4 = 10$

$A[4] = 10 + 5 = 15$

$A[5] = 15 + 6 = 21$

$A[6] = 21 + 7 = 28$

	0	1	2	3	4	5	6
Initial Array	1	2	3	4	5	6	7
	0	1	2	3	4	5	6
Step 1	1	3					
	0	1	2	3	4	5	6
Step 2	1	3	6				
	0	1	2	3	4	5	6
Step 3	1	3	6	10			
	0	1	2	3	4	5	6
Step 4	1	3	6	10	15		
	0	1	2	3	4	5	6
Step 5	1	3	6	10	15	21	
	0	1	2	3	4	5	6
Step 6	1	3	6	10	15	21	28

7.10 Exercise

what is the return value of fun(345,10)

```
int fun(int n, int r)
```

```
{
    if(n>0)
        return ((n%r)+fun(n/r,r));
    else
        return 0;
}
```

Output: 12

What is the return value of fun(16384) and also calculate time complexity-

```
int fun(int n)
{
    if(n<=2)
        return 1;
    else
        return (fun(floor(sqrt(n)))+n);
}
```

Output:

Fun(16384)=16527

Time complexity= $O(\sqrt{n})$

Multiple Choice Questions

1	<pre>#include <stdio.h> int counter = 0; int calc (int a, int b) { int c; counter++; if (b==3) return (a*a*a); else { c = calc(a, b/3); return (c*c*c); } } int main (){ calc(4, 81); printf ("%d", counter); }</pre> The output of this program is _____.
A	3
B	2
C	4
D	6
Answer	C
Difficulty	Moderate
Explanation	
Exam Type	GATE 2018

2	Consider the following program written in pseudo-code. Assume that x and y are integers. <pre> Count(x,y) { if (y != 1){ if (x != 1) { printf("*"); Count(x/2, y); } else { y = y-1; Count(1024, y); } } } </pre> The number of times that the print statement is executed by the call Count(1024,1024) is ____.
A	10240
B	10250
C	10230
D	10220
Answer	C
Difficulty	Difficult
Explanation	
Exam Type	GATE 2018

3	What is the output printed by the following program? <pre> #include<stdio.h> int f(int n, int k) { if(n==0) return 0; else if(n%2) return f(n/2, 2*k)+k; else return f(n/2, 2*k)-k; } int main() { printf("%d",f(20,1)); return 0; } </pre>
A	5
B	8
C	9
D	20
Answer	C
Difficulty	Moderate
Explanation	Explanation should be complete in all respects.
Exam Type	Gate 2005

4	Consider the following C function: <pre> int f(int n) { static int r=0; if(n<=0) return 1; if(n>3) { r=n; return f(n-2)+2; } return f(n-1)+r; } </pre> What is the value of f(5)?
A	5
B	7
C	9
D	18
Answer	D
Difficulty	Moderate
Explanation	
Exam Type	Gate 2007

5	Consider the code fragment written in C below: <pre> void f(int n) { if(n<=1) { printf("%d",n); } else { f(n/2); printf("%d",n%2); } } </pre> What does f(173) print?
A	010110101
B	010101101
C	10110101
D	10101101
Answer	D
Difficulty	Easy
Explanation	
Exam Type	Gate 2008

6	Consider the following recursive C function that takes two arguments. <pre>unsigned int foo(unsigned int n, unsigned int r) { if(n>0) return((n%r)+foo(n/r,r)); else return 0; }</pre> What is the return value of the function foo when it is called as foo(345,10)?
A	345
B	12
C	5
D	3
Answer	B
Difficulty	Easy
Explanation	
Exam Type	Gate 2011

7	Consider the following two functions. <pre>void fun1 (int n) { if (n == 0) return; printf ("%d", n); fun2 (n - 2); printf ("%d", n); } void fun2 (int n) { if (n == 0) return; printf ("%d", n); fun1 (++n); printf ("%d", n); }</pre> The output printed when fun1 (5) is called is
A	53423122233445
B	53423120112233
C	53423122132435
D	53423120213243
Answer	A
Difficulty	Difficult
Explanation	Explanation should be complete in all respects.
Exam Type	Gate 2017

8	<pre>int fun(int n) { int x = 1, k; if (n == 1) return x; for (k = 1; k < n; ++k) x = x + fun(k) * fun(n-k); return x; }</pre> The return value of fun(5) is _____
A	50
B	52
C	53
D	51
Answer	D
Difficulty	Difficult
Explanation	Find the value of function 1 time and utilize it for future references.
Exam Type	Gate 2015

9	What will be the output of following code? <pre>void get (int n) { if (n < 1) return; get (n-1); get (n-3); printf("%d", n); }</pre> If get(6) function is being called in main() then how many times will the get() function be invoked before returning to the main()?
A	27
B	26
C	25
D	22
Answer	C
Difficulty	Moderate
Explanation	
Exam Type	Gate 2015

10	Consider the following three C functions <pre> [P1] int * g (void) { int x = 10; return (&x); } [P2] int * g (void) { int * px; *px = 10; return px; } [P3] int *g (void) { int *px; px = (int *) malloc (sizeof(int)); *px = 10; return px; } </pre> Which of the above three functions are likely to cause problems with pointers?
A	Only P3
B	Only P1 and P3
C	Only P1 and P2
D	P1, P2 and P3
Answer	C
Difficulty	Moderate
Explanation	In P1, pointer variable x is a local variable to g(), and g() returns pointer to this variable. x may vanish after g() has returned as x exists on stack. So, &x may become invalid. In P2, pointer variable px is being assigned a value without allocating memory to it. P3 works perfectly fine. Memory is allocated to pointer variable px using malloc(). So, px exists on heap, it's existence will remain in memory even after return of g() as it is on heap.
Exam Type	Gate 2001
11.	Consider the following program: <pre> int f(int *p, int n) { if (n <= 1) return 0; else return max(f(p+1,n-1),p[0]-p[1]); } </pre>

	<pre> int main() { int a[] = {3,5,2,6,4}; printf("%d", f(a,5)); } </pre> <i>Note: max(x,y) returns the maximum of x and y.</i> The value printed by this program is _____.
A	4.0
B	1.0
C	2.0
D	3.0
Answer	D
Difficulty	Moderate
Explanation	Explanation should be complete in all respects.
Exam Type	Gate 2016
12.	Consider the following function written in the C programming languages. <pre> void foo(char *a) { if(*a && *a != ' ') { foo(a+1); putchar(*a); } } </pre> The output of the above function on input "ABCD EFGH" is
A	ABCD EFCH
B	ABCD
C	HGFE DCBA
D	DCBA
Answer	D
Difficulty	Moderate
Explanation	
Exam Type	Gate 2015
13.	<pre> #include<stdio.h> int f1(void); int f2(void); int f3(void); </pre>

	<pre> int x = 10; int main () { int x = 1; x += f1 () + f2 () + f3() + f2(); printf ("%d", x); return 0; } int f1 () { int x = 25; x++; return x;} int f2 () { static int x = 50; x++; return x;} int f3 () {x *= 10; return x}; The output of the program is _____. </pre>
A	225
B	235
C	220
D	230
Answer	D
Difficulty	Moderate
Explanation	
Exam Type	Gate 2015
14.	What is the return value of fun(16384) and also calculate time complexity- <pre> int fun(int n) { if(n<=2) return 1; else return (fun(floor(sqrt(n)))+n); } </pre>
A	Fun(16384)=16526 Time complexity=O(n)
B	Fun(16384)=16527 Time complexity=O(√n)
C	Fun(16384)=16527 Time complexity=O(n)
D	Fun(16384)=16526 Time complexity=O(√n)
Answer	B
Difficulty	Difficult
Explanation	
Exam Type	--

15.	What will be the output of following code? <pre> #include <stdio.h> #include <string.h> void fun1(char *x, char *y) { char temp; temp = *x; *x = *y; *y = temp; } void fun(char *a, int l, int r) { int i; if (l == r) printf("%s\n", a); else { for (i = l; i <= r; i++) { fun1((a+l), (a+i)); fun(a, l+1, r); fun1((a+l), (a+i)); } } } int main() { char str[] = "ABC"; int n = strlen(str); fun(str, 0, n-1); return 0; } </pre>
A	ABC ACB BAC BCA CBA

B	ABC ACB BAC
C	ABC ACB BAC BCA CBA CAB
D	None of these
Answer	ABC ACB BAC BCA CBA CAB
Difficulty	Difficult
Explanation	
Exam Type	--
16.	What will be the output of following code? <pre> #include<stdio.h> main() { int n,i; n=f(6); printf("%d",n); } f(int x) { if(x==4) return 3; else { printf("+"); f(x-1); } } </pre>

A	+++3
B	++++3
C	++3
D	+3
Answer	C
Difficulty	Moderate
Explanation	Explanation should be complete in all respects.
Exam Type	--
Answer	D
Difficulty	Easy, Moderate and Difficult
Explanation	Explanation should be complete in all respects.
Exam Type	GATE 2016
17.	Consider the C program below. <pre> #include <stdio.h> int *A, stkTop; int stkFunc(int opcode, int val) { static int size = 0, stkTop = 0; switch (opcode) { case -1: size = val; break; case 0: if(stkTop < size) A[stkTop++] = val; break; default: if (stkTop) return A[-stkTop]; } return -1; } int main() { int B[20]; A = B; stkTop = -1; stkFunc(-1, 10); stkFunc(0, 5); stkFunc(0, 10); printf("%d\n", stkFunc(1,0)+stkFunc(1,0)); } </pre> The value printed by the above program is _____.
A	12
B	13
C	14
D	15

Answer	D
Difficulty	Difficult
Explanation	
Exam Type	GATE 2015
18.	What will be the output of the following C program ? <pre> void foo(int n, int sum) { int k=0 , j=0; if (n==0) return ; k = n%10 ; j= n%10 ; sum = sum + k ; foo(j, sum); printf("%d",k); } int main() { int a = 2048 , sum = 0 ; foo (a, sum); printf("%d", sum); return 0; } </pre>
A	2,0,8,4,0
B	2,0,4,8,0
C	2,0,4,0,8
D	2,4,8,0,0
Answer	B
Difficulty	Easy
Explanation	
Exam Type	GATE 2005

19	Consider the following recursive C function that takes two arguments. <pre> unsigned int foo(unsigned int n, unsigned int r) { if(n>0) return((n%r)+foo(n/r,r)); else return 0; } </pre> What does the above code do?
A	Print n in reverse order
B	Print r in reverse order
C	Print r in order of occurrence

D	Print sum of all digits present in n
Answer	D
Difficulty Level	Moderate
Explanation	
Exam Type	

20	Consider the code fragment written in C below: <pre>void f(int n) { if(n<=1) { printf("%d",n); }else { f(n/2); printf("%d",n%2); } }</pre> What does the above code do?
A	Print remainder in reverse order
B	Print equivalent binary representation of input number
C	Print original number as it is
D	Print remainder only when even number occur
Answer	A and B
Difficulty Level	Moderate
Explanation	Explanation should be complete in all respects.
Exam Type	

Q21. What will be the output of the following code?

```
f(int x)
{
    if(x > 0)
    {
        f(--x);
        printf("%d", x);
        f(--x);
    }
}
```

```
int main()
{
    int a=2;
```

```

        f(a);
        return 0;
    }

```

A	01	B	012
C	12	D	120

Q22. What will be the output of the following code?

```

int fun(int n, int k)
{
    if (n == 0)
        return 0;
    else if(n % 2)
        return fun(n/2, 2*k) + k;
    else
        return fun(n/2, 2*k) - k;
}

```

```

int main()
{
    printf("%d", fun(10, 1));
    return 0;
}

```

A	9	B	5
C	12	D	Error

Q23. What will be the output of the following code?

```

#include <stdio.h>
int fun(int*,int);
int main(void)
{
    int a[]={12,7,13,4,11,6};
    printf("%d",fun(a,6));
    return 0;
}
int fun(int *a,int n)
{
    if(n<=0)
        return(0);
}

```

```
else if(*a%2==0)
    return(*a+fun(a+1,n-1));
else
    return(*a-fun(a+1,n-1));
}
```

A	20		B	15
C	12		D	Error